

Data Flow Diagram Document

ReqAgent — AI Requirement Intake & Tracker

Author: Steven Saji Paul | Role: Business Systems Analyst

Version: 1.0 | Date: June 2026 | Status: Final

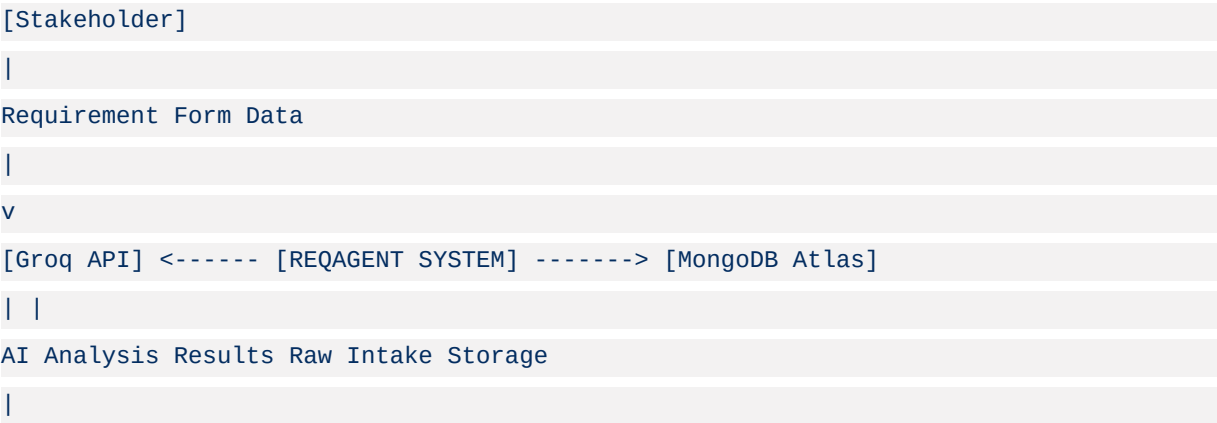
1. Overview

This document describes the data flows within the ReqAgent system. It maps how data moves from external entities through processes into data stores and back to external entities.

2. External Entities

Entity	Type	Description	Data Received
Stakeholder	External	Business user submitting requirements	AI analysis results
Business Analyst	External	Reviews and manages requirements	Tracker, matrix, analytics
Groq API	External System	LLM processing service	JSON with stories, MoSCoW, conflicts
MongoDB Atlas	Data Store	Raw document storage	Existing requirement titles
Neon Postgres	Data Store	Structured data storage	Requirement records, audit log

3. Level 0 — Context Diagram



v

[Neon Postgres]

|

Structured Data Storage

|

v

[Business Analyst]

(Tracker / Matrix / Analytics)

4. Level 1 — Process Decomposition

P1 — Intake Form Processing

Field	Value
Input	Requirement form fields from Stakeholder
Process	Validate required fields, construct document object
Output	Validated requirement document
Error	Validation failure message returned to Stakeholder

P2 — MongoDB Storage

Field	Value
Input	Validated requirement document from P1
Process	Insert document into raw_intake collection
Output	MongoDB ObjectId (mongo_id)
Data Store	MongoDB Atlas — raw_intake collection

P3 — Conflict Data Retrieval

Field	Value
Input	mongo_id from P2
Process	Query MongoDB for all existing requirement titles and descriptions
Output	List of existing requirements for conflict comparison
Data Store	MongoDB Atlas — raw_intake collection (read)

P4 — AI Processing

Field	Value
Input	Requirement text + existing requirements list from P3
Process	Construct structured prompt, call Groq API, parse JSON response
Output	User stories, acceptance criteria, MoSCoW, priority, conflicts
External	Groq API (LLaMA 3.3 70B)

P5 — Postgres Storage

Field	Value
Input	AI output from P4 + mongo_id from P2
Process	Insert into requirements, user_stories, conflicts tables
Output	requirement_id (req_id)
Data Store	Neon Postgres — requirements, user_stories, conflicts tables

P6 — Status Management

Field	Value
Input	Status update from Business Analyst
Process	Update requirements table, insert audit log entry
Output	Updated status + audit trail entry
Data Store	Neon Postgres — requirements, status_audit tables

P7 — Traceability Matrix Generation

Field	Value
Input	BA filter selections
Process	JOIN requirements + user_stories tables, apply filters
Output	Filtered matrix data + CSV file
Data Store	Neon Postgres — requirements, user_stories tables (read)

P8 — Analytics Processing

Field	Value
Input	BA navigates to Analytics tab
Process	Aggregate requirement data by status, priority, MoSCoW, department, role
Output	KPI metrics + chart data
Data Store	Neon Postgres — requirements table (read)

5. Data Dictionary

Data Element	Source	Format	Description
requirement_form_data	Stakeholder	JSON object	All form fields submitted by stakeholder
raw_intake_document	P2 -> MongoDB	BSON document	Complete unstructured intake with metadata
mongo_id	MongoDB	String (ObjectId)	24-character hex ID linking databases
existing_requirements	MongoDB -> P4	Array of objects	Titles and descriptions for conflict check
ai_response	Groq API	JSON object	Stories, MoSCoW, priority, justification, conflicts
req_id	Postgres	Integer	Auto-incremented requirement identifier
status_update	BA -> P6	String	One of 6 lifecycle status values
audit_entry	P6 -> Postgres	Row	old_status, new_status, timestamp, req_id
matrix_data	P7 -> BA	DataFrame	REQ ID, title, stories, criteria, status, priority
analytics_data	P8 -> BA	Aggregated	Counts by status, priority, MoSCoW, department